

Report

South Korea

1 INTRODUCTION

Various tree structures are used for implementing the efficient indexing in the database in many systems. In this report, we implemented three tree data structures (B-Tree, B⁺-Tree, B^{*}-Tree) used for database indexing. Also, we conducted an integrity check and various timing benchmarks for insert, lookup, range operation, and erase with different order of the trees.

Throughout this report we follow the CLRS[1] minimum-degree convention t , where a non-root node holds between $t - 1$ and $2t - 1$ keys. Our implementation exposes the fan-out order d (Knuth's convention, the maximum number of children per node) as the configurable external parameter, and internally converts it to $t = \lceil \frac{d}{2} \rceil$. Because CLRS's parameterization can only express even fan-outs ($d = 2t$), an odd input d rounds up to $d + 1$ (e.g., $d = 5$ yields $t = 3$ with effective max children 6). Throughout the experiments, the reported tree order d corresponds to an internal minimum degree of $t = \lceil d/2 \rceil$.

2 Fundamental Index Structure (B-tree)

2.1 B-Tree

A B-tree is a tree data structure that have following properties[1]:

- Each node x have a $x.n$ keys and the key is stored in monotonically increasing order.
- Each internal node x have $x.n + 1$ pointers to its children.
- The i -th key of the node x $x.key_i$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root i -th child of x , then $k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \dots \leq x.key_{x.n} \leq k_{x.n+1}$
- All leaves have the same depth.
- There is a t for tree which indicates the minimum degree of the tree.
 - Every node other than the root must have at least $t - 1$ keys. Therefore all internal nodes other than the root should have at least t children.
 - Every node may contain at most $2t - 1$ keys. Therefore, an internal node may have at most $2t$ children.

2.1.1 Search. The *Search* operation finds the key in the node which is greater or equal to the target key and if the key is not equal, goes to the child node and do *Search* recursively. The pseudocode is given in Algorithm 1.

2.1.2 Insert. *Insert* performs the same descent as *Search*, but preemptively invokes *Split-Child* whenever the child it is about to descend into is full ($2t - 1$ keys); this guarantees INSERT-NONFULL never recurses into a node that cannot accommodate one more key. The root, having no parent, is handled separately: a full root is wrapped in a new internal node and split, which is the only mechanism by which a B-tree grows in height. Algorithm 2 shows the procedure.

Algorithm 1 B-Tree Search Algorithm

```
1: procedure SEARCH(node, key)
2:    $i \leftarrow 0$ 
3:   while  $i < |node.keys|$  and  $key > node.keys[i]$  do
4:      $i \leftarrow i + 1$ 
5:   end while
6:   if  $i < |node.keys|$  and  $key = node.keys[i]$  then
7:     return (node,  $i$ )
8:   end if
9:   if node.is_leaf then
10:    return NIL
11:  end if
12:  return SEARCH(node.children[ $i$ ], key)
13: end procedure
```

2.1.3 Erase. The *Erase* operation finds the key by descending the tree similar to *Search*. The action depends on where the key is found and the cardinality of the relevant child:

- **Key in a leaf node:** simply remove the (key, value) pair from the leaf.
- **Key in an internal node,** left child has $\geq t$ keys: replace the key with its in-order predecessor (the rightmost key in the left subtree), then recursively erase that predecessor from the left subtree.
- **Key in an internal node, right child has $\geq t$ keys:** symmetric to above, using the in-order successor.
- **Key in an internal node, both children at exactly $t - 1$ keys:** merge the two children together with the separator key (forming a single child with $2t - 1$ keys), then recursively erase from the merged child.
- **Key not in the current node:** descend into the appropriate child. Before descent, if that child has $\leq t - 1$ keys, call *Fill* to bring it to $\geq t$ keys (by *Rotate-Right*, *Rotate-Left*, or *Merge-Two* with a sibling). This pre-condition mirrors the preemptive *Split* in *Insert*: the parent ensures the descended child can survive one more deletion without violating the $t - 1$ minimum.

The full procedure is given in Algorithm 3.

Algorithm 2 B-Tree Insert Algorithm

```

1: procedure INSERT( $T, key, value$ )
2:   if  $T.root = NIL$  then
3:      $T.root \leftarrow$  new leaf node with  $(key, value)$ ; return
4:   end if
5:   if  $|T.root.keys| = 2t - 1$  then
6:      $new\_root \leftarrow$  new internal node
7:      $new\_root.children.push\_back(T.root)$ 
8:     SPLIT-CHILD( $new\_root, 0$ )
9:      $T.root \leftarrow new\_root$ 
10:  end if
11:  return INSERT-NONFULL( $T.root, key, value$ )
12: end procedure

13: procedure INSERT-NONFULL( $node, key, value$ )
14:   $i \leftarrow$  index of key in node
15:  if  $i < |node.keys|$  and  $key = node.keys[i]$  then
16:     $node.values[i] \leftarrow value$ ; return
17:  end if
18:  if  $node.is\_leaf$  then
19:    Insert  $(key, value)$  at position  $i$ ; return
20:  end if
21:  if  $|node.children[i].keys| = 2t - 1$  then
22:    SPLIT-CHILD( $node, i$ )
23:    if  $key > node.keys[i]$  then
24:       $i \leftarrow i + 1$ 
25:    end if
26:  end if
27:  return INSERT-NONFULL( $node.children[i], key, value$ )
28: end procedure

29: procedure SPLIT-CHILD( $parent, i$ )
30:   $full\_node \leftarrow parent.children[i]$ 
31:   $sibling \leftarrow$  new node with same  $is\_leaf$  value
32:   $sibling.keys \leftarrow full\_node.keys[t..2t - 2]$ 
33:   $sibling.values \leftarrow full\_node.values[t..2t - 2]$ 
34:  if not  $full\_node.is\_leaf$  then
35:     $sibling.children \leftarrow full\_node.children[t..2t - 1]$ 
36:  end if
37:   $mid\_k \leftarrow full\_node.keys[t - 1]$ 
38:   $mid\_v \leftarrow full\_node.values[t - 1]$ 
39:  truncate  $full\_node$  to its left half
40:  insert  $sibling$  into  $parent.children[i + 1]$ 
41:  insert  $(mid\_k, mid\_v)$  into  $parent$  at  $i$ 
42: end procedure

```

Algorithm 3 B-Tree Erase Algorithm

```

1: procedure ERASE( $T, key$ )
2:   if  $T.root = NIL$  then
3:     return
4:   end if
5:   DO-ERASE( $T.root, key$ )
6:   if  $|T.root.keys| = 0$  then
7:      $old \leftarrow T.root$ 
8:     if  $old.is\_leaf$  then
9:        $T.root \leftarrow NIL$ 
10:    else
11:       $T.root \leftarrow old.children[0]$ 
12:    end if
13:    free  $old$ 
14:  end if
15: end procedure

16: procedure DO-ERASE( $node, key$ )
17:   $i \leftarrow$  index of key in node
18:  if  $i < |node.keys|$  and  $key = node.keys[i]$  then
19:    if  $node.is\_leaf$  then
20:      remove  $(node.keys[i], node.values[i])$ 
21:    else
22:      ERASE-INTERNAL( $node, i$ )
23:    end if
24:    return
25:  end if
26:  if  $node.is\_leaf$  then
27:    return
28:  end if
29:   $last \leftarrow (i = |node.keys|)$ 
30:  if  $|node.children[i].keys| \leq t - 1$  then
31:    FILL( $node, i$ )
32:  end if
33:  if  $last$  and  $i > |node.keys|$  then
34:    DO-ERASE( $node.children[i - 1], key$ ); return
35:  end if
36:  DO-ERASE( $node.children[i], key$ )
37: end procedure

38: procedure ERASE-INTERNAL( $node, i$ )
39:   $L \leftarrow node.children[i]$ ;  $R \leftarrow node.children[i + 1]$ 
40:  if  $|L.keys| \geq t$  then
41:     $(k, v) \leftarrow$  rightmost  $(key, value)$  in  $L$ 
42:     $node.keys[i] \leftarrow k$ ;  $node.values[i] \leftarrow v$ 
43:    DO-ERASE( $L, k$ )
44:  else if  $|R.keys| \geq t$  then
45:     $(k, v) \leftarrow$  leftmost  $(key, value)$  in  $R$ 
46:     $node.keys[i] \leftarrow k$ ;  $node.values[i] \leftarrow v$ 
47:    DO-ERASE( $R, k$ )
48:  else
49:     $key \leftarrow node.keys[i]$ 
50:    MERGE-TWO( $node, i$ )
51:    DO-ERASE( $node.children[i], key$ )
52:  end if
53: end procedure

54: procedure FILL( $parent, i$ )
55:  if  $i > 0$  and  $|parent.children[i - 1].keys| \geq t$  then
56:    ROTATE-RIGHT( $parent, i$ )
57:  else if  $i < |parent.keys|$  and  $|parent.children[i + 1].keys| \geq t$  then
58:    ROTATE-LEFT( $parent, i$ )
59:  else

```

3.1.4 Range Query. The *Range* operation performs an in-order traversal bounded by $[low, high]$, descending into a child's subtree before visiting each key whose value falls in the range. Algorithm 4 gives the pseudocode.

3 Advanced Index Structure (B+-tree, B*-tree)

3.1 B⁺-Tree

A B⁺-tree shares the structural invariants of a B-tree (Section 2.1) but differs in two key ways: (1) all satellite data (record pointers) is stored only in leaf nodes, while internal nodes hold only routing keys; and (2) leaf nodes are linked as a doubly-linked list to support efficient sequential access. Internal-node routing uses strict inequalities — if k is in the subtree of the i -th child then $k < x.key_i$, and equal keys descend to the right.

3.1.1 Search. Since B⁺-Tree contains the data only in the leaf nodes, it first finds the leaf node contains the key and search the leaf node iteratively. The pseudocode is given in Algorithm 5.

Algorithm 4 B-Tree Range Query

```

1: procedure RANGE(node, low, high, visit)
2:    $i \leftarrow 0$ 
3:   while  $i < |node.keys|$  and  $node.keys[i] < low$  do
4:      $i \leftarrow i + 1$ 
5:   end while
6:   while  $i < |node.keys|$  and  $node.keys[i] \leq high$  do
7:     if not node.is_leaf then
8:       RANGE(node.children[ $i$ ], low, high, visit)
9:     end if
10:    VISIT(node.keys[ $i$ ], node.values[ $i$ ])
11:     $i \leftarrow i + 1$ 
12:   end while
13:   if not node.is_leaf then
14:     RANGE(node.children[ $i$ ], low, high, visit)
15:   end if
16: end procedure

```

Algorithm 5 B⁺ Tree Search Algorithm

```

1: procedure SEARCH(T, key)
2:   leaf  $\leftarrow$  FIND-LEAF(T.root, key)
3:    $i \leftarrow$  first index such that  $leaf.keys[i] \geq key$ 
4:   if  $i < |leaf.keys|$  and  $leaf.keys[i] = key$  then
5:     return (leaf,  $i$ )
6:   end if
7:   return NIL
8: end procedure
9: procedure FIND-LEAF(node, key)
10:  while not node.is_leaf do
11:     $i \leftarrow$  first index such that  $key < node.keys[i]$ 
12:    node  $\leftarrow$  node.children[ $i$ ]
13:  end while
14:  return node
15: end procedure

```

Algorithm 6 B⁺ Tree Insert Algorithm

```

1: procedure SPLIT-LEAF-CHILD(parent,  $i$ )
2:   full_node  $\leftarrow$  parent.children[ $i$ ]
3:   sibling  $\leftarrow$  new leaf node
4:   sibling.keys  $\leftarrow$  full_node.keys[ $t \dots 2t - 1$ ]
5:   sibling.values  $\leftarrow$  full_node.values[ $t \dots 2t - 1$ ]
6:   truncate full_node to first  $t$  keys
7:   LINK-LEAF-LIST(full_node, sibling)
8:   router  $\leftarrow$  sibling.keys[0]
9:   insert sibling into parent.children at position  $i + 1$ 
10:  insert router into parent.keys at position  $i$ 
11: end procedure
12: procedure SPLIT-INTERNAL-CHILD(parent,  $i$ )
13:    $\triangleright$  Same logic with Split-Child in B-Tree
14: end procedure

```

Algorithm 7 B⁺ Tree Erase Algorithm

```

1: procedure DO-ERASE(node, key)
2:   if node.is_leaf then
3:     if key is found in node then
4:       remove (key, value)
5:     end if
6:     return
7:   end if
8:    $i \leftarrow$  first index such that  $key < node.keys[i]$ 
9:   if  $|node.children[i].keys| \leq t - 1$  then
10:     $i \leftarrow$  FILL(node,  $i$ )
11:   end if
12:   DO-ERASE(node.children[ $i$ ], key)
13: end procedure

```

3.1.2 Insert. The *Insert* operation of the B⁺-Tree have the same method as B-Tree (Algorithm 2). However, the *Split* for leaf node is little different. It doesn't remove the mid key from the leaf node. Instead, it copied from the leaf node to parent node and serve as router in the parent node. *Split* for internal node is same as the normal B-Tree. The leaf split procedure is shown in Algorithm 6.

3.1.3 Erase. The *Erase* method follows the same logic as B-Tree, but with two simplifications: actual key removal only occurs at leaf nodes, and the case-analysis of B-tree's ERASE-INTERNAL is unnecessary because internal keys are mere routing copies. Stale internal routing keys are tolerated since they still correctly partition the subtrees. MERGE-TWO is identical to B-tree's, except that when merging two leaves it additionally updates the leaf-list pointer ($L.next \leftarrow R.next$). Algorithm 7 shows the modified DO-ERASE.

3.1.4 Range. Since B⁺-Tree have a links between leaf nodes and actual data is only contained in the leaf nodes, the *Range* leverages this property. *Range* operation finds the leaf node with the low key and iteratively traverse the leaf nodes instead of in-order traversal in B-Tree. The pseudocode is given in Algorithm 8.

Algorithm 8 B+ Tree Range Query

```

1: procedure RANGE( $T, low, high, visit$ )
2:    $leaf \leftarrow \text{FIND-LEAF}(T.root, low)$ 
3:    $i \leftarrow$  first index such that  $leaf.keys[i] \geq low$ 
4:   while  $leaf \neq \text{NIL}$  do
5:     for  $j \leftarrow i$  to  $|leaf.keys| - 1$  do
6:       if  $leaf.keys[j] > high$  then
7:         return
8:       end if
9:       VISIT( $leaf.keys[j], leaf.values[j]$ )
10:    end for
11:     $leaf \leftarrow leaf.next$ 
12:     $i \leftarrow 0$ 
13:  end while
14: end procedure

```

3.2 B*-Tree

A B*-tree is a variant of the B-tree introduced by Knuth[2] that aims for higher average node occupancy. While a standard B-tree guarantees $\geq t - 1$ keys per non-root node ($\approx 1/2$ full), a B*-tree targets approximately $2/3$ occupancy by deferring splits.

3.2.1 Insert Policy. A B*-tree shares the same structural invariants as a B-tree (same capacity, leaf-depth, routing semantics); only the split policy on insertion differs. When a child becomes full, the algorithm first attempts **cooperative redistribution**: if an adjacent sibling has spare capacity, keys are redistributed between them and the parent's separator is updated, allocating no new node. Only when both adjacent siblings are also full does the algorithm fall back to a $2 \rightarrow 3$ **split**: the full child and one full sibling (a combined $4t - 3$ keys including the separator) are repartitioned into three nodes of approximately $\lceil (4t - 3)/3 \rceil$ keys each, promoting two new separators. Newly created nodes thus start at $\approx 2/3$ capacity instead of B-tree's $1/2$. Algorithm 9 shows the modified DO-SPLIT; REDISTRIBUTE-WITH-LEFT is the symmetric mirror of REDISTRIBUTE-WITH-RIGHT and is omitted.

3.2.2 Erase Policy. Maintaining a strict $2/3$ minimum on *erase* would require a $3 \rightarrow 2$ merge mechanism that is mathematically impossible at certain corner cases (e.g., a non-leaf root with two children both at the strict minimum, whose combined size exceeds capacity). Real-world implementations such as HFS+ and NTFS therefore fall back to B-tree's erase semantics, and we follow the same practical choice: *Erase*, *Erase-Internal*, and *Fill* are inherited unchanged from B-tree (Algorithm 3). The $\approx 2/3$ occupancy is thus a statistical tendency from insertions, not a strict invariant maintained on deletion.

3.3 Time Complexity Comparison

For all three trees, the height is $O(\log_b n)$ and per-node key search is $O(t)$ (linear) or $O(\log t)$ (binary), giving Search/Insert/Erase costs of $O(t \log_b n)$ or $O(\log t \log_b n)$ respectively. The B+ tree's Search always descends to a leaf even when an internal routing key matches, which adds only a constant factor. Range queries cost $O(\log_b n + k)$ asymptotically for all three; however, the B+ tree leaf-list scan touches $\lceil k/B \rceil$ contiguous nodes (where B is leaf occupancy), whereas the

Algorithm 9 B* Tree Split and Redistribution

```

1: procedure DO-SPLIT( $parent, i$ )
2:    $M \leftarrow 2t - 1$ 
3:   if  $i > 0$  and  $|parent.children[i - 1].keys| \leq M - 2$  then
4:     REDISTRIBUTE-WITH-LEFT( $parent, i$ )
5:     return
6:   else if  $i < |parent.keys|$  and  $|parent.children[i + 1].keys| \leq M - 2$  then
7:     REDISTRIBUTE-WITH-RIGHT( $parent, i$ )
8:     return
9:   end if
10:  if  $i < |parent.keys|$  then
11:    SPLIT-TWO-INTO-THREE( $parent, i$ )
12:  else if  $i > 0$  then
13:    SPLIT-TWO-INTO-THREE( $parent, i - 1$ )
14:  else
15:    SPLIT-IN-TWO( $parent, i$ )
16:  end if
17: end procedure
18: procedure REDISTRIBUTE-WITH-RIGHT( $parent, i$ )
19:    $L \leftarrow parent.children[i]$ 
20:    $R \leftarrow parent.children[i + 1]$ 
21:    $K \leftarrow L.keys ++ [parent.keys[i]] ++ R.keys$ 
22:    $V \leftarrow$  analogous for values
23:    $new\_L \leftarrow \lfloor |K|/2 \rfloor$ 
24:    $L.keys, L.values \leftarrow K[0 \dots new\_L - 1]$ 
25:    $parent.keys[i], parent.values[i] \leftarrow K[new\_L]$ 
26:    $R.keys, R.values \leftarrow K[new\_L + 1 \dots ]$ 
27:    $\triangleright$  Children pointers redistributed analogously when not
      leaves
28: end procedure

```

Algorithm 9 B* Tree Split and Redistribution (Continued)

```

1: procedure SPLIT-TWO-INTO-THREE( $parent, i$ )
2:    $L \leftarrow parent.children[i]$ 
3:    $R \leftarrow parent.children[i + 1]$ 
4:    $K \leftarrow L.keys ++ [parent.keys[i]] ++ R.keys$ 
5:    $V \leftarrow$  analogous for values
6:    $keys\_to\_distribute \leftarrow |K| - 2$ 
7:    $a \leftarrow \lceil keys\_to\_distribute/3 \rceil$ 
8:    $b \leftarrow \lceil (keys\_to\_distribute - a)/2 \rceil$ 
9:    $M\_node \leftarrow$  new node
10:   $L.keys \leftarrow K[0 \dots a - 1]$ 
11:   $sep1 \leftarrow K[a]$ 
12:   $M\_node.keys \leftarrow K[a + 1 \dots a + b]$ 
13:   $sep2 \leftarrow K[a + b + 1]$ 
14:   $R.keys \leftarrow K[a + b + 2 \dots ]$ 
15:   $parent.keys[i] \leftarrow sep1$ 
16:  insert  $sep2$  into  $parent.keys$  at position  $i + 1$ 
17:  insert  $M\_node$  into  $parent.children$  at position  $i + 1$ 
18: end procedure

```

B-tree and B⁺-tree in-order recursion additionally visits $O(k/t)$ internal nodes — a constant-factor advantage for B⁺-tree on cache- or disk-bound workloads. The B^{*}-tree Insert keeps the same asymptotic bound, but each split on the path performs cooperative redistribution or $2 \rightarrow 3$ split at a $\Theta(t)$ cost — roughly a 2–3× constant-factor overhead compared to B-tree’s simple $1 \rightarrow 2$ split, traded for higher fill factor.

4 Experimental Results and Analysis

4.1 Experiment Setup

We run the insertion of 100000 datas in given csv file into the tree and indexing of 100000 keys, which consists of inserted key in the given data and random keys that may not be in the data. Also, we run the range operations in various range sizes where it calculates the sum of the 1000x of the GPA of all students in given range. Moreover, we performed the erase of 50000 random keys in the tree. During the erase, it checks whether the tree violates the property of the trees such as node key size and same height property. We used a average data of 10 repetitions. To remove a difference caused by the cache and other system issues, we run the 2 times of workload before the actual run. The environment is WSL2 on the Windows 11 with Intel(R) Core(TM) Ultra 7 258V CPU and 16GB RAM. The code is compiled with `-std=c++17 -fsanitize=address -O2` flags. The raw data in text and csv form and implementation code is available at GitHub repository: https://github.com/YOUR_USERNAME/YOUR_REPOSITORY_NAME.

4.2 Insertion and Parameter Tuning



Figure 1: Performance graph for inserting 100000 datas

As shown in fig. 1, for the insertion, B^{*}-Tree shows the worst performance in all orders. B-Tree and B⁺-Tree shows a insignificant differences on their performance. With the respect to the orders, the performance is increasing as the order grows until the order reaches at some point around 100, after which the performance decreases.

Unlike B-tree and B⁺-tree, which perform a simple $1 \rightarrow 2$ node split upon overflow, the B^{*}-tree delays the split by attempting to redistribute keys to adjacent siblings to maintain a $2/3$ minimum occupancy. If the siblings are also full, it performs a complex $2 \rightarrow 3$ split, which causes significant memory copy overhead and frequent cache misses during the insertion phase.

The U-shaped performance curve across different orders illustrates the trade-off between tree height and intra-node operations. As the order d increases up to approximately 100, the tree height $O(\log_d n)$ decreases logarithmically. However, as the order continues to grow beyond the sweet spot ($d > 100$), the size of the internal sorted arrays becomes excessively large. Inserting the keys in such large nodes causes many memory move overhead, which overshadows the benefits of a shallower tree, leading to performance drop.

4.3 Indexing and Parameter Tuning



Figure 2: Performance graph for searching 100000 datas

All trees shows a similar performances, except for $d = 4$, where the B^{*}-Tree performance is the worst. B^{*}-tree exhibits noticeably higher lookup latency compared to the other trees. This becomes the strength of the B^{*}-Tree because it reduces the memory usage. However, in case of small degree, the CPU can fit the node within a single CPU cache line, the I/O benefits of the higher fill factor are removed. Instead, extra intra-node comparison cycles results the performance decreases.

It also shows U-shaped performance curve across different orders illustrates the trade-off between tree height and intra-node operations. The reason of U-shape is same as insertion.

4.4 Range query and Tree types



Figure 3: Performance graph for range query

Note that y-axis of Figure 3 is log scaled.

B+-tree performs the best in almost every d values while other two tree performance is similar. This shows the property of the B+-tree which contains the datas only in the leaf nodes and links between leaf nodes. B+-tree range operation utilizes this property to achieve faster range query in sequential manner, while other trees needs to switch between parent node and child node during range query.

The U-shaped performance curve doesn't showed in this graph. This is because range queries perform read-only sequential scans, completely bypassing the expensive $O(d)$ memory shifting operations required for node modifications. Moreover, larger d makes more keys are packed into a single node, which utilizes the CPU cache operations during range operation.

4.5 Erase and Integrity



Figure 4: Performance graph for erase

As shown in fig. 4, the deletion performance forms a U-shaped curve, penalised by $O(d)$ memory shifting overhead at large orders. However, an interesting inversion occurs between the B*-tree and B+-tree around $d = 16$.

On the small degree, B+-Tree utilizes the property that it contains the actual data only at the leaf node which allows the bypass of *Erase-Internal*, which contains the time consuming operation such as find successors. However, on large tree, although the B*-tree inherits its erase algorithm directly from the standard B-tree without a strict $3 \rightarrow 2$ merge invariant, its superior performance stems from its initial structural density. Because B*-tree nodes hove around 66% density, the other keys performs as 'Extra Buffer' which prevents underflow. Underflow needs a huge structural modifications such as merge, rotate, and borrow. By statistically avoiding expensive structural modifications (such as node rotations and merges), the B*-tree achieves the most efficient deletion times at larger orders.

For the integrity, we ran the check internal operation which check the property of the tree such as monotonically increasing keys, number of keys and children, height of leaf nodes, links on the leaf nodes in B+-Tree. The check operation ran periodically during the erasing keys and it results zero violations. Specifically, the internal check function was called after every 5000 deletions (10% of the workload) Also, we checked the cross-check between reference STL map. After the completion of workload, the total size, remaining keys, values are compared with STL map implementation, ensuring the implementation is algorithmically sound and robust.

5 Additional Experiments: Insert order, Fill factor, and Height

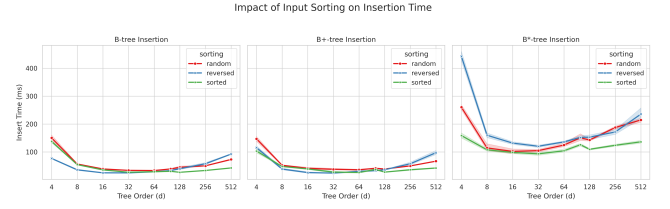


Figure 5: Performance graph for various input order

Additionally, the input sorting drastically affects performance. Sorted inputs append to the rightmost leaf, achieving $O(1)$ cost and maximizing cache locality. Conversely, reversed inputs strictly insert at index 0, forcing an $O(d)$ memmove for every insertion, which critically degrades performance. At $d = 4$ with reversed inputs, the B*-tree suffers severe bottlenecks as continuous index-0 insertions trigger relentless redistributions under its strict 66% fill-factor constraint.

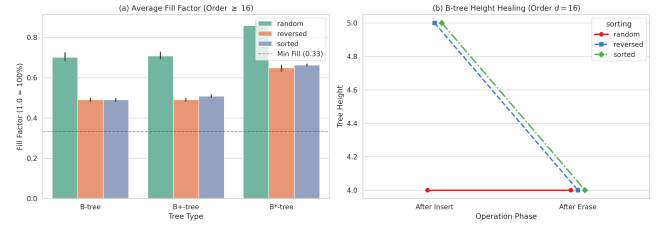


Figure 6: Fill factor and height difference

To deeply understand the B-tree variants' internal behavior, we extracted and analyzed the node fill factors and tree heights during the workloads. As illustrated in fig. 6, input sorting directly dictates spatial efficiency. Reversed insertions cause pathological 33% fill factors in B/B+-trees (one key per node), artificially inflating the tree height due to abandoned right-side nodes. Interestingly, subsequent random deletions trigger internal MERGE operations, successfully "healing" the sparse nodes and compressing the tree back to its optimal height. The B*-tree, however, rigidly maintains its height and a superior $> 80\%$ fill factor under random inputs, proving its structural efficiency.

References

- [1] Thomas H. Cormen, Charles Eric Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to algorithms*. The MIT Press.
- [2] Donald E. Knuth. 1998. *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.